

1. Design a divide and conquer algorithm for polynomial evaluation.
(For convenience assume that the polynomial is of degree $(n-1)$ and $n=2m$ where m is a positive integer). Calculate the number of additions and multiplications required by your algorithm.

Divide; Polinomu tek ve çift gücüne göre ayır

$$A_{\text{even}}(x) = a_0 + a_2x + a_4x^2 + \dots + a_{n/2-2}x^{(n-1)/2}$$

$$A_{\text{odd}}(x) = a_1 + a_3x + a_5x^2 + \dots + a_{n/2-1}x^{(n-1)/2}$$

$$A(x) = A_{\text{even}}(x^2) + x A_{\text{odd}}(x^2)$$

Conquer: $\frac{1}{2}$ kökte A_{even} 'i ve A_{odd} 'u hesapla; $v_0, v_1, \dots, v_{n/2-1}$

Combine: $A(w^k) = A_{\text{even}}(v^k) + w^k A_{\text{odd}}(v^k) \quad 0 \leq k < n/2$

$$A(w^{k+n/2}) = A_{\text{even}}(v^k) - w^k A_{\text{odd}}(v^k) \quad 0 \leq k < n/2$$

Algorithm Polynomial Evaluation ($n, a_0, a_1, \dots, a_{n-1}$)

if ($n = 1$) return a_0

$(e_0, e_1, e_2, \dots, e_{n/2-1}) \leftarrow \text{Algorithm Polynomial Evaluation}(n/2, a_0, a_1, \dots, a_{n/2})$

$(d_0, d_1, d_2, \dots, d_{n/2-1}) \leftarrow \text{Algorithm Polynomial Evaluation}(n/2, a_{n/2}, a_{n/2+1}, \dots, a_{n-1})$

for $k=0$ to $n/2-1$

$$y_k \leftarrow e_k + w^k d_k$$

$$y_{k+n/2} \leftarrow e_k - w^k d_k$$

return $(y_0, y_1, \dots, y_{n-1})$

end

2. Given an array $A[1 \dots n]$ of sorted distinct integers, design a divide and conquer algorithm that finds an index i such that $A[i] = i$. Your algorithm should run in $O(\log n)$ time.

```
FindAnIndex ( A[], low, high, element) ;  
    if low ≤ high :  
        middle ← low + high / 2  
        if element == A[middle] :  
            if element == middle :  
                return element  
        else if element < A[middle] :  
            return FindAnIndex(A, low, middle - 1, element)  
        else :  
            return FindAnIndex(A, middle + 1, high, element)  
    return -(low + 1)
```

4. Write a pseudocode for a divide-and-conquer algorithm for finding values of both the largest and smallest elements in an array of n numbers.

Algorithm minmax ($A[l \dots r]$, minVal, maxVal)

if $r = l$

minVal $\leftarrow A[l]$

maxVal $\leftarrow A[l]$

else if $r - l = 1$

if $A[l] \leq A[r]$,

minVal $\leftarrow A[l]$

maxVal $\leftarrow A[r]$

else

minVal $\leftarrow A[r]$

maxVal $\leftarrow A[l]$

else

minmax ($A[l \dots \lfloor (l+r)/2 \rfloor]$, minVal, maxVal)

minmax ($A[\lfloor (l+r)/2 \rfloor + 1 \dots r]$, minVal2, maxVal2)

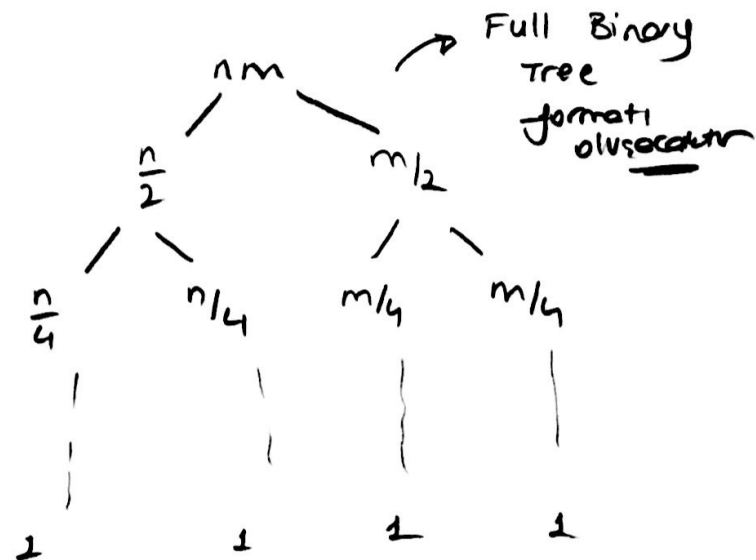
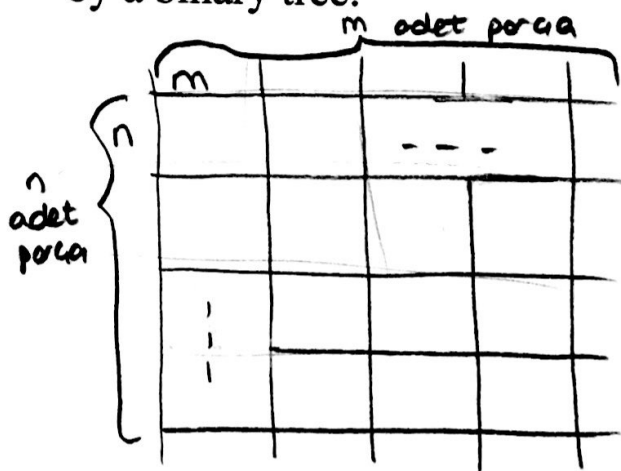
if minVal2 $<$ minVal

minVal $\leftarrow$ minVal2

if maxVal2 $>$ maxVal

maxVal $\leftarrow$ maxVal2

5. You are given a chocolate bar puzzle given as an n -by- m chocolate bar. You need to break it into nm 1-by-1 pieces. You can break a bar only in a straight line, and only one bar can be broken at a time. Design an algorithm that solves the problem with the minimum number of bar breaks. What is this minimum number? Justify your answer by using properties of a binary tree. Breaking the chocolate bar can be represented by a binary tree.



$$C(n) = 2n + 1$$

$$n = nm - 1 \Rightarrow 2(nm - 1) + 1$$

$$= 2nm - 2 + 1$$

$$= \underline{\underline{2nm - 1}}$$

tekil
parçalar
ayrılacak
4 tane bölünebilir
çakıl
gsterilebilir
 $\Theta(nm)$

6. Questions are given below. Please answer them in detail. a. What does dynamic programming have in common with divide-and conquer?
b. What is the principal difference between the two techniques?

Çalışma Prensipleri :

Divide and Conquer ;

Divide - Problem daha küçük parçalara ve alt problemlere ayrılır.

Conquer - Recursive olarak şekilde altprogramlara ayrıldıktan sonra her alt program için ayrı sonuçlar içeren çözümler bulunur.

Combine - Her alt programa ait sonuçlar birleştirilir ve orijinal problemin genel sonucu oluşturulmuş olur.

Dynamic Programming;

→ Ana problem küçük parçalara bölünür.

→ Her altproblem sadece bir kez çözülür, (Recursive kullanılmaz)

→ Tutulan sonuçlar bir önceki probleme ait sonuçları tutar.

(Altproblemlerin sonuçları bir tabloda keyler yardımıyla tutulur)

→ Tekrar tekrar alt problemlerin sonuçları hesaplanmak zorunda kalmaz

→ Sıra ile birleştirilir.

a) Ortak Yönleri:

→ İki yaklaşımda da problem kendinden daha küçük alt problemlere bölünür.

→ Sonrasında bu alt problemleri çözeriz.

→ Sonrasında büyük problemi çözebilmek için her alt problemin çözümünü birleştiririz.

→ Çözüm derinliklere sahip altproblemlere parçalanabiliriz.

b) Farklı Yönleri:

→ Divide and conquer algoritmalarında recursive çözümler bulunur fakat Dynamic Programming'de tasarlanan algoritma recursive yapıda değildir.

→ Divide and conquer'da altproblemlerin sonuçları ayrı ayrı tutulur. Dynamic programming'de farklı veri yapıları oluşturularak sonuçlar orada tutularak sıra sıra ana problemin çözümü yapılır.

→ Tüm divide and conquer algoritmalarında fazladan yer alması ihtiyacı yoktur. Fakat tüm dynamic programming algoritmaları fazla boyutlarda space alınarak oluşturulmuş yapılar tutulur.

Farklı yöntemlere ek olarak;

→ Divide and conquer yönteminde alt problemlerin bağımsız olması gerekmektedir.

→ Dynamic programming'de alt problemler bağımsız değilse de, Dynamic Programming uygulanabilir.

→ Dynamic programming her alt problemi bir kez ister ve çözümleri bir tabloda saklar.

→ Divide and conquer algoritmalarında alt problemin birden fazla çözümleri durumunda her seferinde tekrar çözüm yapmak gerekir. Fakat dynamic programming'de her bir değer tabloda saklandığı için tekrar çözüm yapmaya gerek yoktur. Problemlerde direkt olarak kullanılır. Bu şekilde daha hızlı çözümlenmiş olur.

7. *World Series odds*: Consider two teams, A and B , playing a series of games until one of the teams wins n games. Assume that the probability of A winning a game is the same for each game and equal to p , and the probability of A losing a game is $q = 1 - p$. (Hence, there are no ties.) Let $P(i, j)$ be the probability of A winning the series if A needs i more games to win the series and B needs j more games to win the series.

a. Set up a recurrence relation for $P(i, j)$ that can be used by a dynamic programming algorithm. b. Find the probability of team A winning a seven-game series if the probability of it winning a game is 0.4.

c. Write a pseudocode of the dynamic programming algorithm for solving this problem and determine its time and space efficiencies.

① A'nın kazanacağı toplam oyun sayısı $\rightarrow i$ olarak ifade edilmiş
 B'nin " " " " " " $\rightarrow j$ olarak ifade edilmiş
 Toplam oyun sayısı olasılığı $P(i, j)$ olarak ifade edilmiş
 A'nın kazandığı her oyunun olasılığı p ile ifade edilirken, B'nin kazandığı her oyun için olasılık q ile ifade edilir.

$$p + q = 1$$

A oyun kazanırsa toplam oyun sayısından 1 eksilecek, B'nin oyun sayısı değişmeyecektir.

$$P(i-1, j) \cdot p$$

↑
A'nın oyun kazanma olasılığı

Ters durumda da;

$$P(i, j-1) \cdot q$$

↑
B'nin oyun kazanma olasılığı

Recurrence Relation: $P(i, j) = pP(i-1, j) + qP(i, j-1)$

$$P(0, j) = 1$$

$$P(i, 0) = 0$$

b) $P(0, j) = 1$ $p(i, 0) = 0$ ve

$P(i, j) = pP(i-1, j) + qP(i, j-1)$ durumuna göre

$i \backslash j$	0	1	2	3	4
0		1	1	1	1
1	0	0,4	0,64	0,78	0,87
2	0	0,16	0,35	0,52	0,66
3	0	0,06	0,18	0,32	0,46
4	0	0,03	0,09	0,18	<u>0,29</u>

Initial condition

A'nın kazanma durumu 0,4 olarak verilmiştir

$P(1,1) = pP(0,1) + qP(1,0) = 0,4$

$P(1,2) = pP(0,2) + qP(1,1) = 0,24 + 0,40 = 0,64$

$P(1,3) = pP(0,3) + qP(1,2) = 0,78$

$P(1,4) = pP(0,4) + qP(1,3) = 0,87$

$P(2,1) = pP(1,1) + qP(2,0) = 0,16$

$P(2,2) = pP(1,2) + qP(2,1) = 0,35$

$P(2,3) = pP(1,3) + qP(2,2) = 0,52$

$P(2,4) = pP(1,4) + qP(2,3) = 0,66$

$P(3,1) = pP(2,1) + qP(3,0) = 0,06$

$P(3,2) = pP(2,2) + qP(3,1) = 0,18$

$P(3,3) = pP(2,3) + qP(3,2) = 0,32$

$P(3,4) = pP(2,4) + qP(3,3) = 0,46$

$P(4,1) = pP(3,1) + qP(4,0) = 0,03$

$P(4,2) = pP(3,2) + qP(4,1) = 0,09$

$P(4,3) = pP(3,3) + qP(4,2) = 0,18$

$P(4,4) = pP(3,4) + qP(4,3) = 0,29$

c) Procedure WorldSeries (n, p)

$q \leftarrow 1 - p$

for $j \leftarrow 1$ to n do

$P[0, j] \leftarrow 1.0$

for $i \leftarrow 1$ to n do

$P[i, 0] \leftarrow 0.0$

for $j \leftarrow 1$ to n do

$P[i, j] \leftarrow p * P[i-1, j] + q * P[i, j-1]$

return $P[n, n]$

Time Efficiency: $\Theta(n^2)$

Space Efficiency: $\Theta(n^2)$

8. Given a binary matrix, find out the maximum size square sub-matrix with all 1s. For example, consider the below binary matrix.

0	1	1	0	1
1	1	0	1	0
0	1	1	1	0
1	1	1	1	0
1	1	1	1	1
0	0	0	0	0

The maximum square sub- matrix with all set bits is 1.

1	1	1
1	1	1
1	1	1

Result will be 3 as given example above. Propose a dynamic programming algorithm for this problem. Write your code in Python implementing your dynamic programming approach. Explain your algorithm in detail. (Grade will be given according to your algorithm and output of your program.)

Best Case

→ Sadece 1 satırı olan matrislerin içinde bulunan "1" değeri square sub matrix olup boyutu da 1 olarak değerlendirilir.

→ Sadece 1 sütunu olan matrislerin içinde bulunan "1" değeri square sub matrix olup boyutu da 1 olarak değerlendirilir.

Dynamic Programming İzlene Matrisi;

→ Verilen dizi ile aynı boyutta yardımcı dizi oluşturulur. Yardımcı dizi belirli noktalara göre mümkün olan en büyük kare matrisi değeri ile doldurulur.

→ Yardımcı matris doldurulduktan sonra, maximum değeri olan hücre bulunur. Bu 1'lerle doldurulan maximum karenin boyutuna ulaşılmaz, sepet.

Yardımcı matris Nasıl Doldurulur?

→ İlk satır ile ilk sütunu Best Case'e göre doldururuz.

→ Gelen değer ;

0 ise

0 hücreye
"0" değeri
kaydırız

1 ise

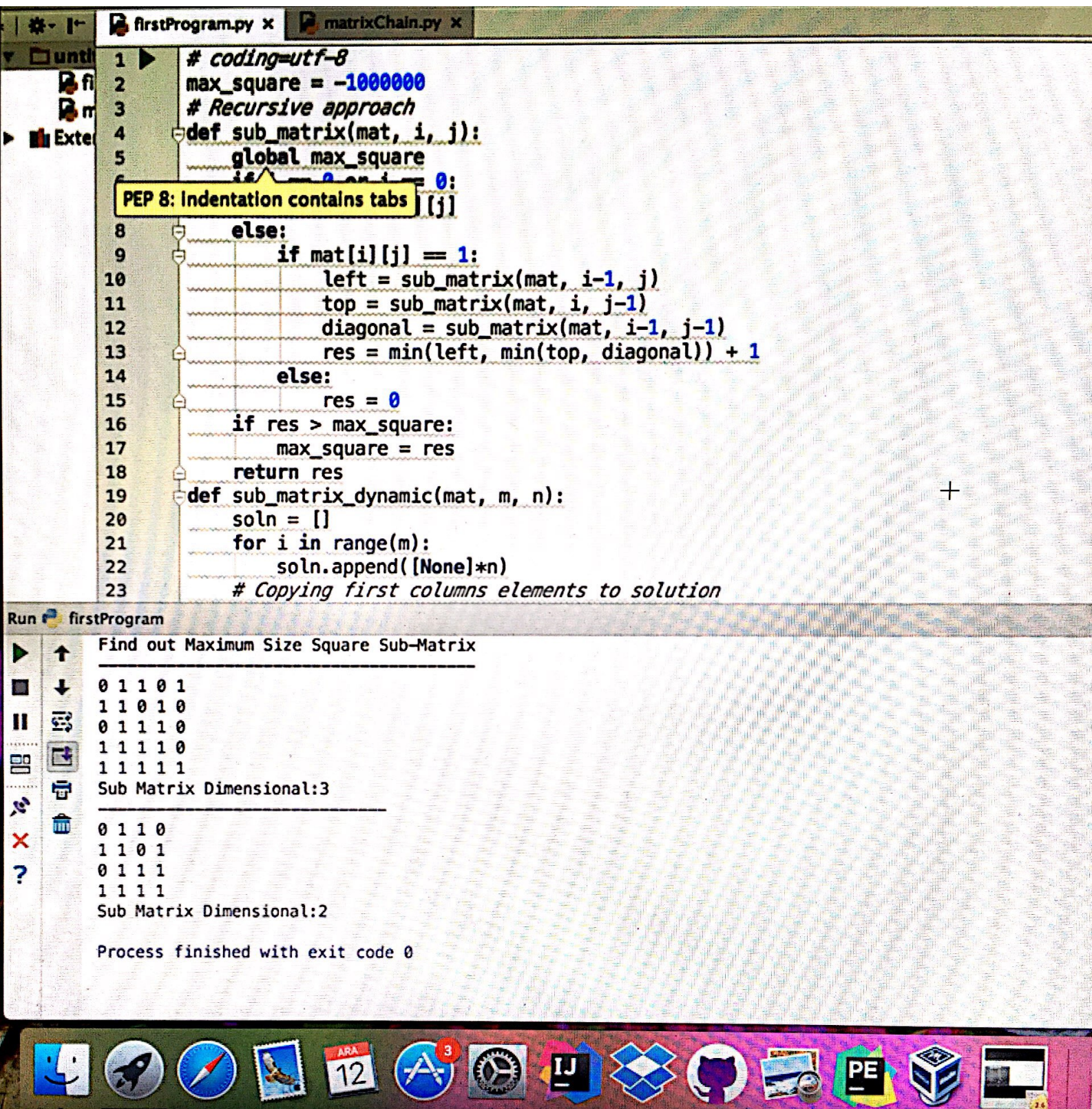
Ak, Üst, Sol, Sağ
diğerlerine bakarak
1 kaydırız.


```

# coding=utf-8
max_square = -1000000
# Recursive approach
def sub_matrix(mat, i, j):
    global max_square
    if i == 0 or j == 0:
        return mat[i][j]
    else:
        if mat[i][j] == 1:
            left = sub_matrix(mat, i-1, j)
            top = sub_matrix(mat, i, j-1)
            diagonal = sub_matrix(mat, i-1, j-1)
            res = min(left, min(top, diagonal)) + 1
        else:
            res = 0
        if res > max_square:
            max_square = res
        return res
def sub_matrix_dynamic(mat, m, n):
    soln = []
    for i in range(m):
        soln.append([None]*n)
    # Copying first columns elements to solution
    for i in range(m):
        soln[i][0] = mat[i][0]
    # Copying first row elements from matrix
    for j in range(n):
        soln[0][j] = mat[0][j]
    for i in range(1, m):
        for j in range(1, n):
            if mat[i][j] == 1:
                soln[i][j] = min(soln[i-1][j], min(soln[i-1][j-1], soln[i][j-1])) + 1
            else:
                soln[i][j] = 0
    maximum = soln[0][0]
    for i in range(m):
        for j in range(n):
            if soln[i][j] > maximum:
                maximum = soln[i][j]
    return maximum

```

PEP 8: indentation contains tabs



Zincir Matris Grubu:

$\langle A_1, A_2, \dots, A_n \rangle$ matris zinciri olsun. $C = A_1 A_2 \dots A_n$ grubu hesaplanarak isteniyorsa, Matris Grubunun birleşme özelliği vardır.

$$\langle A_1, A_2, A_3, A_4 \rangle$$

$$(A_1 (A_2 (A_3 A_4)))$$

$$(A_1 (A_2 A_3) A_4)$$

$$((A_1 A_2) (A_3 A_4))$$

$$(A_1 (A_2 A_3) A_4)$$

$$(((A_1 A_2) A_3) A_4)$$

Seçilecek olan
parantezlenmenin
hesaplama
maliyeti
üzerinde önemli
bir etkisi vardır.

1. Alternatif

$$A_1 \rightarrow 10 \times 100 \quad B \rightarrow 100 \times 5 \quad C \rightarrow 5 \times 50 \rightarrow \underbrace{p \times q}_{p \times q}, \underbrace{q \times r}_{p \times q}$$

Bu üç matrisin grubu $((A_1 A_2) A_3)$ parantezlenmesine göre yapılırsa, A_1, A_2 matrislerinin grubunda $10 \cdot 100 \cdot 5 = 500$ tone skaler grup yapılır. Elde edilen sonuç matrisin boyutları 10×5 olur. Bu matris ile A_3 matrisinin grubu içinde $10 \cdot 5 \cdot 50 = 2500$ tone skaler grup yapılır. Toplam olarak 7500 tone skaler grup yapılır.

2. Alternatif

Eğer matrisler $(A_1 (A_2 A_3))$ şeklinde gruplarsa

$$A_2 \text{ ve } A_3 \text{ iken } \rightarrow 10 \cdot 5 \cdot 50 = 25000 \text{ tone skaler gruba ihtiyacı vardır}$$

Elde edilen sonuç matrisinin boyutları 100×50 olur. Bu matris ile A_1 matrisinin grubu içinde $10 \cdot 100 \cdot 50 = 50000$ tone skaler grup yapılır.

Toplam olarak 75000 tone skaler grup yapılır.

Dynamic Programming ile Zincir Matris Çarpımı:

Çarpılacak $n+1$ matris için yapılacak parantez sayısı $P(n)$ dır.

Bu durumda $P(1) = P(2) = 1$ dır. Ve $P(3)$ için 2, $P(4)$ için 5 dır.

Azaddeci recursive n boyutlu matris için farklı parantezleme sayıları verir.

$$P(n) = \begin{cases} 1 & \text{if } n=1 \\ \sum_{k=1}^{n-1} P(k) P(n-k) & \text{if } n \geq 2 \end{cases}$$

Sıra ve sütunların önemsenmediği sadece kaç farklı gruplandırma yapılabilir bu göstermiştir.

$P(4)$ için

A_1, A_2, A_3, A_4 matrislerimize olsun!!!

$$(A_1 (A_2 A_3 A_4)) \quad ((A_1 A_2) (A_3 A_4)) \quad ((A_1 A_2 A_3) A_4)$$

$$1 + 2 + 1 = 4 \Rightarrow 5$$

Parantezlemenin içindeki matrislerin yerleri önemli değil.

Recursive yapı bu şekilde gösterilebilir.

$P(n) = O(2^n)$ 'dir. Exponential bir büyüme mevcuttur.

Adım Adım Dynamic Programming:

① Optimal Çözümün yapısının karakteristiği ortaya çıkarılmalı;

$A_i A_{i+1} \dots A_j$ çarpımı için aşağıdaki gibi optimal parantezleme yapılabilir.

$$A_i A_{i+1} \dots A_k \quad A_{k+1} \dots A_j \quad 1 \leq i \leq k \leq j \leq n$$

Bu ayrı subproblem'e

② Recursively optimal çözüm değeri ayrıldı.

Minimum Maliyet Recursive

$$m[i, j] = \begin{cases} 0 & \text{if } i=j \\ \min_{i \leq k \leq j} \{ m[i, k] + m[k+1, j] + p_{i-1} p_k p_j \} & \text{if } i < j \end{cases}$$

A_i matrisinin boyutu $p_{i-1} \times p_i$ 'dir.

③ Bottom-up method ile bir optimal çözümleri veren algoritma?

MATRIX-CHAIN-ORDER(p)

$n = p.length - 1$

let $m[1..n, 1..n]$ and $s[1..n-1, 2..n]$ be new tables ^{By tablo 0 boyutlu}

for $i = 1$ to n

$m[i, i] = 0$

for $l = 2$ to n

for $i = 1$ to $n - l + 1$

$j = i + l - 1$

$m[i, j] = \infty$

for $k = i$ to $j - 1$

$q = m[i, k] + m[k+1, j] + p_i - 1 \cdot p_k \cdot p_j$

if $q < m[i, j]$

$m[i, j] = q$

$s[i, j] = k$

return m and s

④ Hesaplama bilgilerden optimal çözümleri elde edilir.

PRINT-OPTIMAL-PARENS(s, i, j)

if $i == j$

print "A"

else

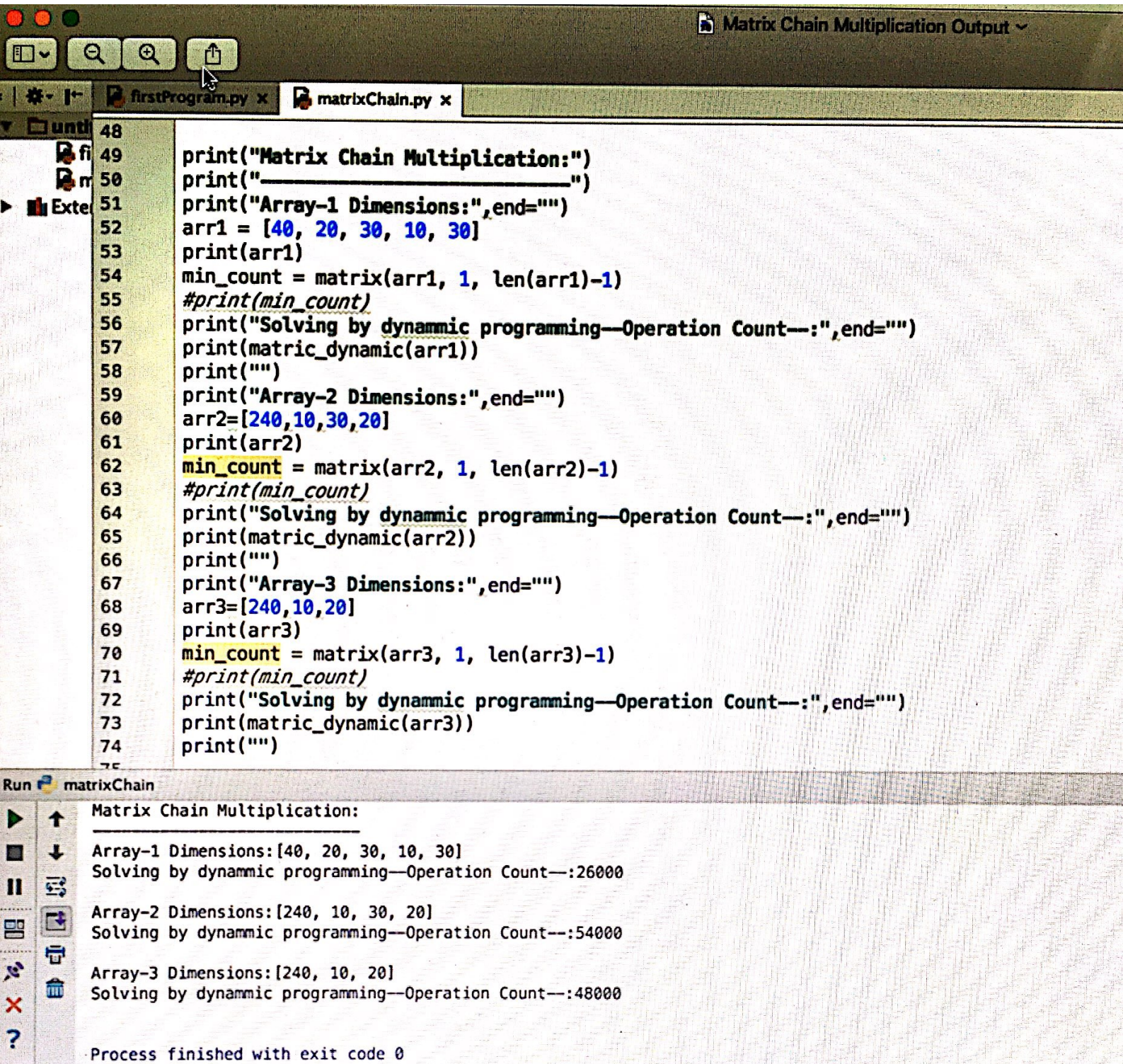
print "("

PRINT-OPTIMAL-PARENS($s, i, s[i, j]$)

PRINT-OPTIMAL-PARENS($s, s[i, j] + 1, j$)

print ")"

for example ile gösterilmeli: PRINT-OPTIMAL-PARENS($s, 1, n$)



The image shows a screenshot of a Python IDE with two tabs: 'firstProgram.py' and 'matrixChain.py'. The 'matrixChain.py' tab is active, displaying a Python script for matrix chain multiplication. The script defines three arrays: arr1 = [40, 20, 30, 10, 30], arr2 = [240, 10, 30, 20], and arr3 = [240, 10, 20]. For each array, it calculates the minimum number of scalar multiplications required to multiply the matrices in the array, using a recursive function 'matrix' and a helper function 'matric_dynamic'. The output of the program is shown in the 'Run' console at the bottom, which displays the dimensions of each array and the resulting operation counts: 26000 for arr1, 54000 for arr2, and 48000 for arr3. The process finished with exit code 0.

```
48 print("Matrix Chain Multiplication:")
49 print("_____")
50
51 print("Array-1 Dimensions:",end="")
52 arr1 = [40, 20, 30, 10, 30]
53 print(arr1)
54 min_count = matrix(arr1, 1, len(arr1)-1)
55 #print(min_count)
56 print("Solving by dynamic programming—Operation Count—:",end="")
57 print(matric_dynamic(arr1))
58 print("")
59 print("Array-2 Dimensions:",end="")
60 arr2=[240,10,30,20]
61 print(arr2)
62 min_count = matrix(arr2, 1, len(arr2)-1)
63 #print(min_count)
64 print("Solving by dynamic programming—Operation Count—:",end="")
65 print(matric_dynamic(arr2))
66 print("")
67 print("Array-3 Dimensions:",end="")
68 arr3=[240,10,20]
69 print(arr3)
70 min_count = matrix(arr3, 1, len(arr3)-1)
71 #print(min_count)
72 print("Solving by dynamic programming—Operation Count—:",end="")
73 print(matric_dynamic(arr3))
74 print("")
75
```

Run matrixChain

Matrix Chain Multiplication:

Array-1 Dimensions:[40, 20, 30, 10, 30]
Solving by dynamic programming—Operation Count—:26000

Array-2 Dimensions:[240, 10, 30, 20]
Solving by dynamic programming—Operation Count—:54000

Array-3 Dimensions:[240, 10, 20]
Solving by dynamic programming—Operation Count—:48000

Process finished with exit code 0

Devrim Arabaları

1950 'lerin Türkiye'sinde Devrim adı verilen otomobilin o zamanın devlet başkanı Cemal Paşa tarafından istenmesi üzerine seçilen mühendisler tarafından yapılması sürecini anlatan filmidir. Filmin ilk sahnesinde bu süreçte yer almış bir mühendisin müzeyi ziyaret etmesi ve o süreci gözünün önüne getirmesi ile başlamaktadır. Başta Türk bakanların araba yapılamaz iktisadi olarak mümkün değildir fikrine karşı gelerek Cemal Paşa'nın isteğini kırmayan Türk Demiryolları başkanı mühendis Gündüz Bey 'in bu fikri kabul ediş süreci gözlenmiştir. Sonrasında proje müdürü olarak başa gelen Gündüz Bey mesleğinde uzman olan mühendisleri toplayarak Eskişehir'deki fabrikada araba yapımına başlanmıştır. Araba yapımı için toplam 130 günleri vardır. 29 Ekim 'e kadar teslim etmeleri gerekmektedir. İlk başta tüm mühendisler bunun yapılamayacağına inanmaktaydı fakat Gündüz Bey'in konuşması ve Amerika'dan yeni gelen mühendis Uğur'un projeksiyonda gösterdiği filmle akıllarında birkaç fikir oluşan mühendisler arabayı yapmayı kabul etmişlerdir. Bu süreç içerisinde kalıp çıkartmışlardır. Piston 'ların yapımına başlamışlardır. Bu süreçte verdikleri örneklere uygun gelen pistonlar istedikleri kuvvete güçsüz kalmıştır daha sonrasında mühendis Necip'in önerisiyle bir demir fabrikasında çalışan Recep Usta onlar için piston yapmış ve gerçekten de tam istedikleri gibi olmuştur. Hiçbir hesap yapmadan direk pistonun boyutlarını söyleyen Recep Usta'ya şaşırmışlardır. Sonrasında Recep Usta' da onlara katılmıştır. Bakanlar ilk günden beri bu araba yapımına karşılardır. Bundan dolayı bütçelerini sınırlandırıp zor durumda bırakmaya çalışarak araba yapımını imkansız hale sürüklemeye çalışmışlardır. Araba yapımını istemeyenlerden biri de gazetecidir. Bu gazeteci halka araba yapanları kötü göstermeye çalışmışlardır. Tüm bu şartlara rağmen Türk mühendisler canla başla savaşmışlardır. 2 ay sonunda Cemal Paşa yapılıp görmek istemiştir. Bunun haberini 3 gün önce duyan mühendisler ne yapacaklarını şaşırmıştır. İlk başta yaptıkları motor çalışmamış ve hepsi ümitsizliğe kapılmıştır. Fakat Gündüz Bey'in konuşması onları tekrardan cesaretlendirmiş ve Cemal Paşa'nın yanında çalışır vaziyete gelmiştir. Daha sonrasında zor durumda kalmalarına rağmen araba yapımı tamamlanmıştır. Fakat sırf benzin yetmediği için araba 29 ekim de yolda kalmıştır. Bunu hemen haber yapan gazetecilerden dolayı bu kadar zahmete katlanmalarına rağmen halk tarafından iyi karşılanmamıştır.